DÖNGÜLER

1. GİRİŞ

Programlarda belirli işlemlerin birden fazla kez tekrarlanması gerekebilir. Örneğin 1'den 100'e kadar sayıları yazdırmak, bir dizinin elemanlarını toplamak veya kullanıcıdan geçerli bir değer alınana kadar giriş istemek. Bu tür tekrarlı işlemler için **döngüler** kullanılır. Döngüler, bir kod bloğunun belirli bir koşul sağlandığı sürece tekrar tekrar çalıştırılmasını sağlar. Bu bölümde `for` ve `while` döngüleri, iç içe döngüler ve sayı problemlerine yönelik uygulamalar C++ dili üzerinden ele alınacaktır.

2. DÖNGÜ NEDİR?

Döngü, belirli bir kod bloğunun bir koşul sağlandığı sürece tekrar tekrar çalıştırılmasını sağlayan kontrol yapısıdır.



Günlük hayattan benzetme:

- "10 kere şınav çek."
- "Saat 17:00 olana kadar çalış."
- "Listede eleman kalmayana kadar işle."

Bu örneklerin her biri birer döngüdür.

3. FOR DÖNGÜSÜ

3.1 For Döngüsünün Yapısı

`for` döngüsü, tekrar sayısı önceden bilinen durumlarda kullanılır.

Genel Yapı:

```
for (başlangıç; koşul; artış) {
    // Tekrarlanacak kodlar
}
```

Üç Bölüm:



3.2 For Döngüsü Akış Şeması



3.3 Basit For Döngüsü Örneği

```cpp
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    cout << endl;

    return 0;
}
```

Çıktı:

```
1 2 3 4 5
```

Adım Adım Çalışma:

```
i = 1  -->  Koşul: 1 <= 5 (Doğru)  -->  Yaz: 1  -->  i = 2
i = 2  -->  Koşul: 2 <= 5 (Doğru)  -->  Yaz: 2  -->  i = 3
i = 3  -->  Koşul: 3 <= 5 (Doğru)  -->  Yaz: 3  -->  i = 4
i = 4  -->  Koşul: 4 <= 5 (Doğru)  -->  Yaz: 4  -->  i = 5
i = 5  -->  Koşul: 5 <= 5 (Doğru)  -->  Yaz: 5  -->  i = 6
i = 6  -->  Koşul: 6 <= 5 (Yanlış) -->  Döngü bitti
```

3.4 For Döngüsünün Farklı Kullanımları

Geriye Doğru Sayma:

```
for (int i = 10; i >= 1; i--) {
    cout << i << " ";
}
// Çıktı: 10 9 8 7 6 5 4 3 2 1
```

İkişer İkişer Artış:

```
for (int i = 0; i <= 10; i += 2) {
    cout << i << " ";
}
// Çıktı: 0 2 4 6 8 10
```

Birden Fazla Değişken:

```
for (int i = 1, j = 10; i <= 5; i++, j--) {
    cout << "i: " << i << ", j: " << j << endl;
}
// Çıktı:
// i: 1, j: 10
// i: 2, j: 9
// i: 3, j: 8
// i: 4, j: 7
// i: 5, j: 6
```

Sonsuz Döngü:

```
for (;;) {
    // Sonsuz döngü
    // break ile çıkılabilir
}
```

Boş Gövdeli Döngü:

```
int i;
for (i = 0; i < 5; i++);  // Noktalı virgül dikkat
cout << i;  // Çıktı: 5
```

3.5 For Döngüsü ile Toplam Hesaplama

```cpp
#include <iostream>
using namespace std;

int main() {
    int toplam = 0;

    for (int i = 1; i <= 100; i++) {
        toplam += i;
    }

    cout << "1'den 100'e kadar toplam: " << toplam << endl;

    return 0;
}
```

Çıktı:

```
1'den 100'e kadar toplam: 5050
```

Adım Adım Çalışma:

```
toplam = 0
i = 1:   toplam = 0 + 1 = 1
i = 2:   toplam = 1 + 2 = 3
i = 3:   toplam = 3 + 3 = 6
...
i = 100: toplam = 4950 + 100 = 5050
```

---

4. WHILE DÖNGÜSÜ

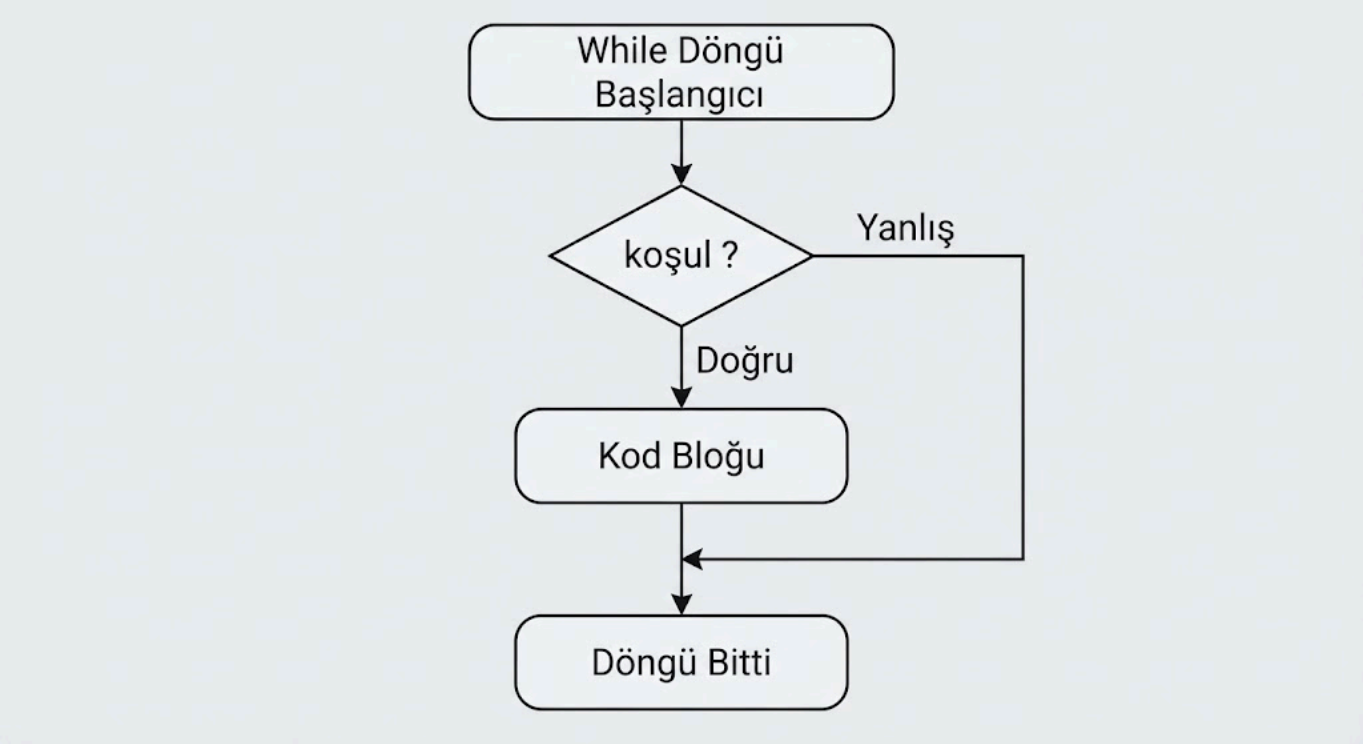
4.1 While Döngüsünün Yapısı

`while` döngüsü, tekrar sayısı önceden bilinmeyen ve bir koşula bağlı olan durumlarda kullanılır.

Genel Yapı:

```cpp
while (koşul) {
    // Tekrarlanacak kodlar
}
```

Akış Şeması:



4.2 Basit While Döngüsü Örneği

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;

    while (i <= 5) {
        cout << i << " ";
        i++;
    }
    cout << endl;

    return 0;
}
```

Çıktı:

```
1 2 3 4 5
```

Adım Adım Çalışma:

```
i = 1  -->  Koşul: 1 <= 5 (Doğru)  -->  Yaz: 1  -->  i = 2
i = 2  -->  Koşul: 2 <= 5 (Doğru)  -->  Yaz: 2  -->  i = 3
...
```

```
i = 5  -->  Koşul: 5 <= 5 (Doğru)  -->  Yaz: 5  -->  i = 6
i = 6  -->  Koşul: 6 <= 5 (Yanlış) -->  Döngü bitti
```

4.3 While Döngüsü ile Kullanıcı Girişi

```
#include <iostream>
using namespace std;

int main() {
    int sayi;

    cout << "Pozitif bir sayı girin: ";
    cin >> sayi;

    while (sayi <= 0) {
        cout << "Geçersiz! yeni sayı girin ";
        cin >> sayi;
    }

    cout << "Girdiğiniz sayı: " << sayi << endl;

    return 0;
}
```

Çalışma Örneği:

```
Pozitif bir sayı girin: -5
Geçersiz! Pozitif bir sayı girin: 0
Geçersiz! Pozitif bir sayı girin: 12
Girdiğiniz sayı: 12
```

4.4 While Döngüsü ile Sayı Tahmin Oyunu

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0));
    int hedef = rand() % 100 + 1;
    int tahmin;
    int deneme = 0;

    cout << "1-100 arası bir sayı tuttum. Tahmin edin!" << endl;

    while (true) {
```

```
        cout << "Tahmininiz: ";
        cin >> tahmin;
        deneme++;

        if (tahmin == hedef) {
            cout << "Doğru! " << deneme << " denemede buldunuz." << endl;
            break;
        } else if (tahmin < hedef) {
            cout << "Daha büyük bir sayı girin." << endl;
        } else {
            cout << "Daha küçük bir sayı girin." << endl;
        }
    }

    return 0;
}
```

Çalışma Örneği:

```
1-100 arası bir sayı tuttum. Tahmin edin!
Tahmininiz: 50
Daha küçük bir sayı girin.
Tahmininiz: 25
Daha büyük bir sayı girin.
Tahmininiz: 37
Doğru! 3 denemede buldunuz.
```

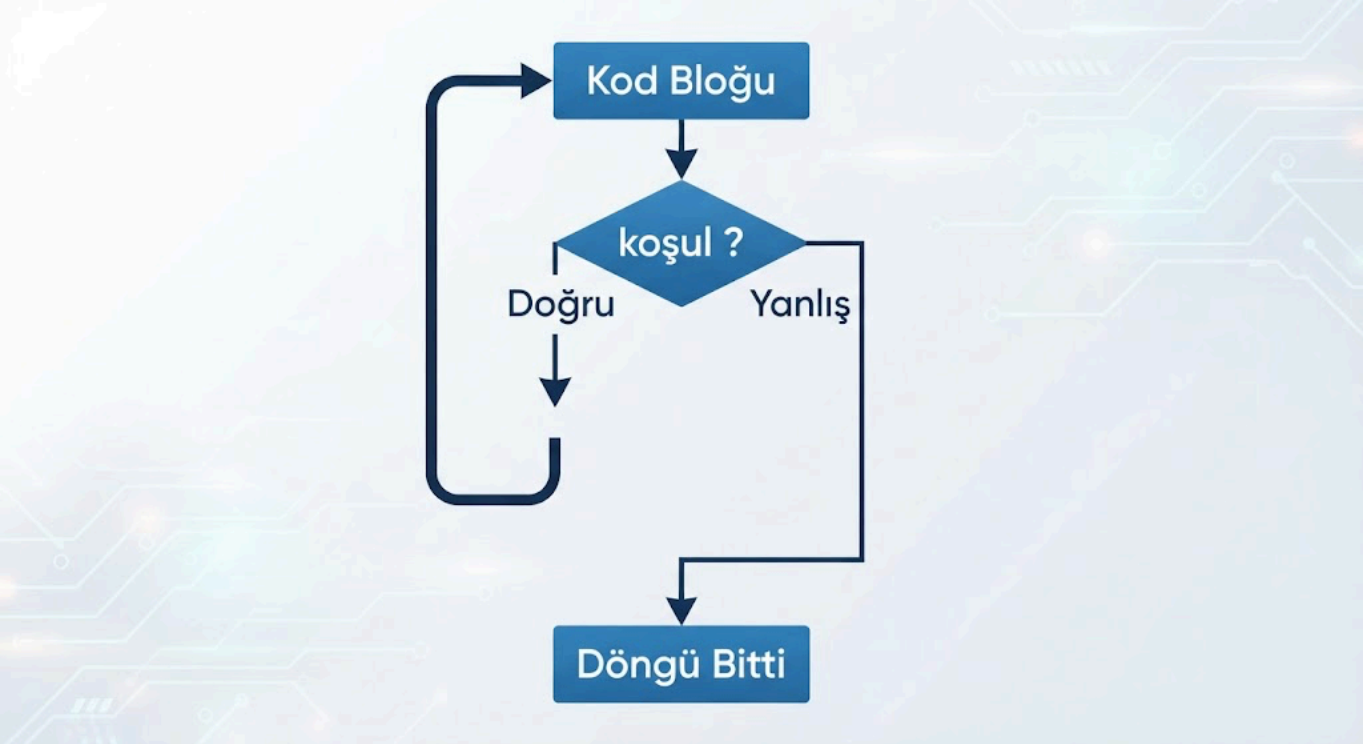
4.5 do-while Döngüsü

`do-while` döngüsü, kod bloğunu **en az bir kez** çalıştırır, sonra koşulu kontrol eder.

Genel Yapı:

```
do {
    // Tekrarlanacak kodlar
} while (koşul);
```

Akış Şeması:



Örnek:

```
#include <iostream>
using namespace std;

int main() {
    int sayi;

    do {
        cout << "Pozitif bir sayı girin: ";
        cin >> sayi;
    } while (sayi <= 0);

    cout << "Girdiğiniz sayı: " << sayi << endl;

    return 0;
}
```

4.6 While vs For Karşılaştırması

```
+----------------------+----------------------+----------------------+
| Özellik              | for                  | while                |
+----------------------+----------------------+----------------------+
| Kullanım Durumu      | Tekrar sayısı bilinen| Tekrar sayısı bilinmez|
| Başlangıç            | Döngü içinde         | Döngü öncesi         |
| Artış                | Döngü içinde         | Döngü içinde         |
| Koşul Kontrolü       | Her turda başta      | Her turda başta      |
| Okunabilirlik        | Sayaçlı işlemler     | Koşullu işlemler     |
```

```
| Sonsuz Döngü         | for(;;)              | while(true)          |
+----------------------+----------------------+----------------------+
```

4.7 Döngü Kontrol Komutları

break: Döngüyü tamamen sonlandırır.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;  // i = 5'te döngüden çıkar
    }
    cout << i << " ";
}
// Çıktı: 1 2 3 4
```

continue: O turu atlar, sonraki tura geçer.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        continue;  // i = 5'i atlar
    }
    cout << i << " ";
}
// Çıktı: 1 2 3 4 6 7 8 9 10
```

break vs continue Karşılaştırması:

```
+----------------------+----------------------------------------+
| Komut                | Etkisi                                 |
+----------------------+----------------------------------------+
| break                | Döngüyü tamamen bitirir                |
| continue             | O turu atlar, sonraki tura geçer       |
+----------------------+----------------------------------------+
```

---

5. İÇ İÇE DÖNGÜLER

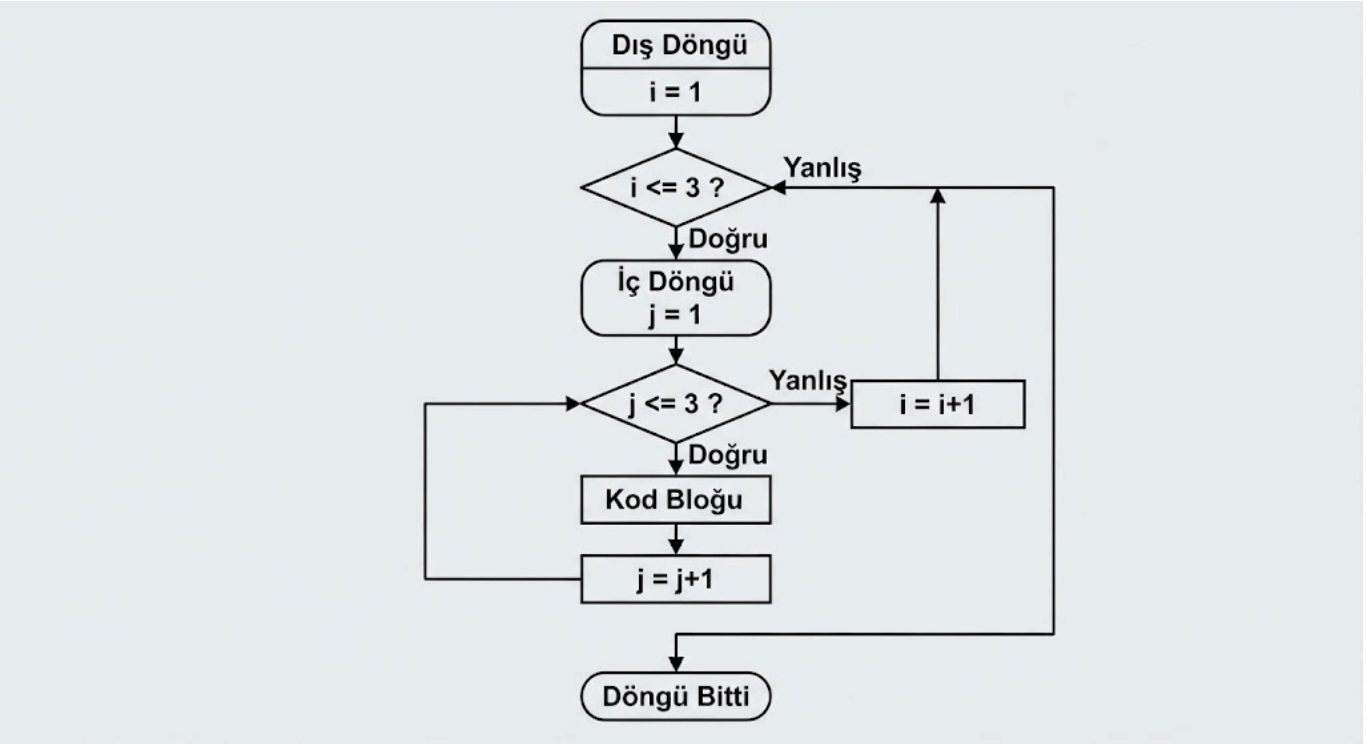
5.1 İç İçe Döngü Nedir?

Bir döngünün içinde başka bir döngünün bulunmasına **iç içe döngü (nested loop)** denir. Genellikle iki boyutlu yapılar (matris, tablo, çarpım tablosu) için kullanılır.

Genel Yapı:

```
for (başlangıç1; koşul1; artış1) {
    for (başlangıç2; koşul2; artış2) {
        // İç döngü kodları
    }
    // Dış döngü kodları
}
```

5.2 İç İçe Döngü Akış Şeması



5.3 Basit İç İçe Döngü Örneği

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            cout << "(" << i << "," << j << ") ";
        }
        cout << endl;
    }

    return 0;
}
```

Çıktı:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

Adım Adım Çalışma:

```
i = 1:
  j = 1:  Yaz (1,1)
  j = 2:  Yaz (1,2)
  j = 3:  Yaz (1,3)
  j = 4:  İç döngü bitti
i = 2:
  j = 1:  Yaz (2,1)
  j = 2:  Yaz (2,2)
  j = 3:  Yaz (2,3)
  j = 4:  İç döngü bitti
i = 3:
  j = 1:  Yaz (3,1)
  j = 2:  Yaz (3,2)
  j = 3:  Yaz (3,3)
  j = 4:  İç döngü bitti
i = 4:  Dış döngü bitti
```

5.4 Çarpım Tablosu

```cpp
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        for (int j = 1; j <= 10; j++) {
            cout << i << "x" << j << "=" << (i * j) << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Çıktı (Kısaltılmış):

```
1x1=1   1x2=2   1x3=3   ...   1x10=10
2x1=2   2x2=4   2x3=6   ...   2x10=20
3x1=3   3x2=6   3x3=9   ...   3x10=30
```

```
...
10x1=10 10x2=20 10x3=30 ...   10x10=100
```

5.5 Yıldızlarla Üçgen Çizimi

Dik Üçgen:

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }

    return 0;
}
```

Çıktı:

```
*
* *
* * *
* * * *
* * * * *
```

Ters Üçgen:

```cpp
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        cout << "* ";
    }
    cout << endl;
}
```

Çıktı:

```
* * * * *
* * * *
* * *
```

```
* *
*
```

Piramit:

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        // Boşluklar
        for (int j = 1; j <= n - i; j++) {
            cout << " ";
        }
        // Yıldızlar
        for (int j = 1; j <= 2 * i - 1; j++) {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}
```

Çıktı:

```
    *
   ***
  *****
 *******
*********
```

5.6 İç İçe Döngülerde Zaman Karmaşıklığı

İç içe döngülerde toplam işlem sayısı, döngülerin çarpımı kadardır.

```
+---------------------+---------------------+---------------------+
| Dış Döngü           | İç Döngü            | Toplam İşlem        |
+---------------------+---------------------+---------------------+
| n kere              | n kere              | n × n = n²          |
| n kere              | m kere              | n × m               |
| n kere              | n kere (iç içe 2)   | n³                  |
+---------------------+---------------------+---------------------+
```

Örnek:

```
// O(n²) karmaşıklık
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // n × n = n² işlem
    }
}
```

5.7 İç İçe Döngülerde Dikkat Edilmesi Gerekenler

```
+-----------------------------------+---------------------------------------
--+
| Kural                             | Açıklama
|
+-----------------------------------+---------------------------------------
--+
| Değişken isimleri farklı olmalı   | i, j, k gibi farklı isimler kullan
|
| İç döngü dış döngüden bağımsız    | İç döngü her dış turda sıfırlanır
|
| break sadece iç döngüyü kırar     | Dış döngüyü kırmak için ek kontrol
|
| Performans düşebilir              | n² veya n³ işlem sayısına dikkat
|
| Gereksiz iç içe döngüden kaçın     | Mümkünse tek döngüyle çöz
|
+-----------------------------------+---------------------------------------
--+
```

---

6. UYGULAMA: SAYI PROBLEMLERİ (C++ DİLİNDE)

6.1 Problem 1: 1'den N'e Kadar Sayıların Toplamı

Problem: Kullanıcıdan alınan N sayısına kadar olan sayıların toplamını bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n, toplam = 0;

    cout << "N değerini girin: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        toplam += i;
```

```
    }

    cout << "1'den " << n << "'e kadar toplam: " << toplam << endl;

    return 0;
}
```

Çalışma Örneği:

```
N değerini girin: 10
1'den 10'e kadar toplam: 55
```

6.2 Problem 2: Faktöriyel Hesaplama

Problem: Kullanıcıdan alınan N sayısının faktöriyelini hesaplayan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long faktoriyel = 1;

    cout << "N değerini girin: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        faktoriyel *= i;
    }

    cout << n << "! = " << faktoriyel << endl;

    return 0;
}
```

Çalışma Örnekleri:

```
Girdi: 5   -->  Çıktı: 5! = 120
Girdi: 10  -->  Çıktı: 10! = 3628800
```

6.3 Problem 3: Çift Sayıların Toplamı

Problem: 1'den 100'e kadar olan çift sayıların toplamını bulan C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int toplam = 0;

    for (int i = 2; i <= 100; i += 2) {
        toplam += i;
    }

    cout << "1-100 arası çift sayıların toplamı: " << toplam << endl;

    return 0;
}
```

Çıktı:

```
1-100 arası çift sayıların toplamı: 2550
```

6.4 Problem 4: Sayıyı Tersten Yazdırma

Problem: Kullanıcıdan alınan sayının basamaklarını tersten yazdıran C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int sayi, ters = 0;

    cout << "Bir sayı girin: ";
    cin >> sayi;

    int gecici = sayi;

    while (gecici > 0) {
        ters = ters * 10 + (gecici % 10);
        gecici /= 10;
    }

    cout << sayi << " sayısının tersi: " << ters << endl;

    return 0;
}
```

Çalışma Örneği:

```
Girdi: 1234
Çıktı: 1234 sayısının tersi: 4321
```

Adım Adım Çalışma:

```
sayi = 1234, ters = 0
gecici = 1234:  ters = 0*10 + 4 = 4,    gecici = 123
gecici = 123:   ters = 4*10 + 3 = 43,   gecici = 12
gecici = 12:    ters = 43*10 + 2 = 432, gecici = 1
gecici = 1:     ters = 432*10 + 1 = 4321, gecici = 0
gecici = 0:     döngü bitti
```

6.5 Problem 5: Basamak Sayısını Bulma

Problem: Kullanıcıdan alınan sayının kaç basamaklı olduğunu bulan C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int sayi, basamak = 0;

    cout << "Bir sayı girin: ";
    cin >> sayi;

    int gecici = sayi;

    if (gecici == 0) {
        basamak = 1;
    } else {
        while (gecici != 0) {
            gecici /= 10;
            basamak++;
        }
    }

    cout << sayi << " sayısı " << basamak << " basamaklıdır." << endl;

    return 0;
}
```

Çalışma Örnekleri:

```
Girdi: 12345  -->  Çıktı: 12345 sayısı 5 basamaklıdır.
Girdi: 7      -->  Çıktı: 7 sayısı 1 basamaklıdır.
```

```
Girdi: 0      -->  Çıktı: 0 sayısı 1 basamaklıdır.
```

6.6 Problem 6: Asal Sayıları Listeleme

Problem: 1'den 100'e kadar olan asal sayıları listeleyen C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "1-100 arası asal sayılar:" << endl;

    for (int sayi = 2; sayi <= 100; sayi++) {
        bool asal = true;

        for (int i = 2; i <= sayi / 2; i++) {
            if (sayi % i == 0) {
                asal = false;
                break;
            }
        }

        if (asal) {
            cout << sayi << " ";
        }
    }
    cout << endl;

    return 0;
}
```

Çıktı:

```
1-100 arası asal sayılar:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

6.7 Problem 7: Fibonacci Dizisi

Problem: İlk N Fibonacci sayısını ekrana yazdıran C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç Fibonacci sayısı gösterilsin? ";
```

```
    cin >> n;

    int a = 0, b = 1, gecici;

    cout << "Fibonacci dizisi: ";

    for (int i = 1; i <= n; i++) {
        cout << a << " ";
        gecici = a + b;
        a = b;
        b = gecici;
    }
    cout << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç Fibonacci sayısı gösterilsin? 10
Fibonacci dizisi: 0 1 1 2 3 5 8 13 21 34
```

6.8 Problem 8: EBOB ve EKOK

Problem: İki sayının EBOB ve EKOK'unu bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int a, b, ebob, ekok;

    cout << "İki sayı girin: ";
    cin >> a >> b;

    int x = a, y = b;

    // EBOB (Öklid algoritması)
    while (y != 0) {
        int gecici = y;
        y = x % y;
        x = gecici;
    }
    ebob = x;

    // EKOK
    ekok = (a * b) / ebob;

    cout << "EBOB: " << ebob << endl;
```

```
    cout << "EKOK: " << ekok << endl;

    return 0;
}
```

Çalışma Örneği:

```
İki sayı girin: 12 18
EBOB: 6
EKOK: 36
```

6.9 Problem 9: Mükemmel Sayı Kontrolü

Problem: Kullanıcıdan alınan sayının mükemmel sayı olup olmadığını bulan C++ programı.

Tanım: Kendisi hariç pozitif bölenlerinin toplamı kendisine eşit olan sayı.

```
#include <iostream>
using namespace std;

int main() {
    int sayi, toplam = 0;

    cout << "Bir sayı girin: ";
    cin >> sayi;

    for (int i = 1; i < sayi; i++) {
        if (sayi % i == 0) {
            toplam += i;
        }
    }

    if (toplam == sayi) {
        cout << sayi << " bir mükemmel sayıdır." << endl;
    } else {
        cout << sayi << " bir mükemmel sayı değildir." << endl;
    }

    return 0;
}
```

Çalışma Örnekleri:

```
Girdi: 6   -->  Çıktı: 6 bir mükemmel sayıdır. (1+2+3=6)
Girdi: 28  -->  Çıktı: 28 bir mükemmel sayıdır. (1+2+4+7+14=28)
Girdi: 10  -->  Çıktı: 10 bir mükemmel sayı değildir.
```

6.10 Problem 10: Armstrong Sayısı Kontrolü

Problem: Kullanıcıdan alınan sayının Armstrong sayısı olup olmadığını bulan C++ programı.

Tanım: Bir sayının basamaklarının küplerinin toplamı kendisine eşitse Armstrong sayısıdır. (Örn: 153 = 1^3 + 5^3 + 3^3)

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int sayi, toplam = 0, basamak = 0;

    cout << "Bir sayı girin: ";
    cin >> sayi;

    int gecici = sayi;

    // Basamak sayısını bul
    while (gecici != 0) {
        gecici /= 10;
        basamak++;
    }

    // Basamakların kuvvetlerini topla
    gecici = sayi;
    while (gecici != 0) {
        int rakam = gecici % 10;
        toplam += pow(rakam, basamak);
        gecici /= 10;
    }

    if (toplam == sayi) {
        cout << sayi << " bir Armstrong sayısıdır." << endl;
    } else {
        cout << sayi << " bir Armstrong sayısı değildir." << endl;
    }

    return 0;
}
```

Çalışma Örnekleri:

```
Girdi: 153  -->  Çıktı: 153 bir Armstrong sayısıdır. (1³+5³+3³=153)
Girdi: 370  -->  Çıktı: 370 bir Armstrong sayısıdır. (3³+7³+0³=370)
Girdi: 100  -->  Çıktı: 100 bir Armstrong sayısı değildir.
```

6.11 Problem 11: Sayı Tahmin Oyunu

Problem: Bilgisayarın tuttuğu sayıyı kullanıcının tahmin etmeye çalıştığı C++ programı.

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0));
    int hedef = rand() % 100 + 1;
    int tahmin, deneme = 0;

    cout << "1-100 arası bir sayı tuttum. Tahmin edin!" << endl;

    do {
        cout << "Tahmininiz: ";
        cin >> tahmin;
        deneme++;

        if (tahmin < hedef) {
            cout << "Daha büyük!" << endl;
        } else if (tahmin > hedef) {
            cout << "Daha küçük!" << endl;
        } else {
            cout << "Doğru! " << deneme << " denemede buldunuz." << endl;
        }
    } while (tahmin != hedef);

    return 0;
}
```

Çalışma Örneği:

```
1-100 arası bir sayı tuttum. Tahmin edin!
Tahmininiz: 50
Daha büyük!
Tahmininiz: 75
Daha küçük!
Tahmininiz: 62
Doğru! 3 denemede buldunuz.
```

6.12 Problem 12: Çarpım Tablosu (İç İçe Döngü)

Problem: 1'den 10'a kadar çarpım tablosunu ekrana yazdıran C++ programı.

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main() {
    cout << "Çarpım Tablosu" << endl;
    cout << "==============" << endl;

    for (int i = 1; i <= 10; i++) {
        for (int j = 1; j <= 10; j++) {
            cout << setw(4) << (i * j);
        }
        cout << endl;
    }

    return 0;
}
```

Çıktı (Kısaltılmış):

```
Çarpım Tablosu
==============
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
...
  10  20  30  40  50  60  70  80  90 100
```

6.13 Problem 13: Yıldızlarla Elmas Çizimi

Problem: Kullanıcıdan alınan N değerine göre yıldızlarla elmas çizen C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Elmasın yarı yüksekliğini girin: ";
    cin >> n;

    // Üst kısım
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++) {
            cout << " ";
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            cout << "*";
        }
        cout << endl;
    }
```

```
    // Alt kısım
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= n - i; j++) {
            cout << " ";
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}
```

Çalışma Örneği (n = 4):

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

6.14 Problem 14: Sayıları Sıralama (Bubble Sort)

Problem: Kullanıcıdan alınan N sayıyı küçükten büyüğe sıralayan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int dizi[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> dizi[i];
    }

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (dizi[j] > dizi[j + 1]) {
                int gecici = dizi[j];
```

```
                dizi[j] = dizi[j + 1];
                dizi[j + 1] = gecici;
            }
        }
    }

    cout << "Sıralı dizi: ";
    for (int i = 0; i < n; i++) {
        cout << dizi[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 5
Sayıları girin:
64 25 12 22 11
Sıralı dizi: 11 12 22 25 64
```

6.15 Problem 15: Girilen Sayıların Ortalaması

Problem: Kullanıcıdan kaç sayı gireceğini alıp, sonra bu sayıların ortalamasını hesaplayan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    double toplam = 0, ortalama;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        double sayi;
        cout << i << ". sayı: ";
        cin >> sayi;
        toplam += sayi;
    }

    ortalama = toplam / n;

    cout << "Ortalama: " << ortalama << endl;
```

```
    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 4
1. sayı: 10
2. sayı: 20
3. sayı: 30
4. sayı: 40
Ortalama: 25
```

---

7. DÖNGÜ TÜRLERİNİN KARŞILAŞTIRMASI

```
+---------------------+---------------------+---------------------+
| Özellik             | for                 | while               |
+---------------------+---------------------+---------------------+
| Tekrar sayısı       | Bilinen             | Bilinmeyen          |
| Koşul kontrolü      | Başta               | Başta               |
| En az çalışma       | 0 kere              | 0 kere              |
| Başlangıç değişkeni | Döngü içinde        | Döngü öncesi        |
| Artış ifadesi       | Döngü içinde        | Döngü içinde        |
| Kullanım kolaylığı  | Sayaçlı işlemler    | Koşullu işlemler    |
+---------------------+---------------------+---------------------+

+---------------------+---------------------+
| Özellik             | do-while            |
+---------------------+---------------------+
| Tekrar sayısı       | Bilinmeyen          |
| Koşul kontrolü      | Sonda               |
| En az çalışma       | 1 kere              |
| Kullanım durumu     | En az bir kez çalışması gereken|
+---------------------+---------------------+
```

---

8. SIK YAPILAN HATALAR

```
+---------------------------------+-------------------------------------
--+
| Hata                            | Doğrusu
|
+---------------------------------+-------------------------------------
--+
| Sonsuz döngü                    | Koşulun bir noktada yanlış olması
|
|                                 | sağlanmalı (artış unutulmamalı)
```

```
|
| Döngü değişkenini dışarıda          | int i; for(i=0;...) yerine
|
| tanımlamak                          | for(int i=0;...) tercih edilmeli
|
| İç içe döngüde aynı değişken        | i, j, k gibi farklı değişkenler
|
| kullanmak                           | kullanılmalı
|
| break/continue yanlış yerde         | Döngü mantığına uygun kullanılmalı
|
| Koşulda = kullanmak                 | == kullanılmalı
|
| Döngü sonrası noktalı virgül        | for(...); yanlış, for(...){} doğru
|
| Güncelleme unutmak                  | i++ gibi artış unutulmamalı
|
+-------------------------------------+-------------------------------------
--+
```

Örnek Hata:

```cpp
// YANLIŞ - Sonsuz döngü
int i = 1;
while (i <= 5) {
    cout << i << " ";
    // i++ unutulmuş!
}

// DOĞRU
int i = 1;
while (i <= 5) {
    cout << i << " ";
    i++;
}
```

---

9. ÖZET

- **Döngüler**, bir kod bloğunun belirli bir koşul sağlandığı sürece tekrar çalıştırılmasını sağlar.
- **for döngüsü**, tekrar sayısı bilinen durumlarda kullanılır; başlangıç, koşul ve artış tek satırda belirtilir.
- **while döngüsü**, tekrar sayısı bilinmeyen ve bir koşula bağlı durumlarda kullanılır.
- **do-while döngüsü**, kod bloğunu en az bir kez çalıştırır, sonra koşulu kontrol eder.
- **break** döngüyü tamamen sonlandırır; **continue** o turu atlar.
- **İç içe döngüler**, iki boyutlu yapılar (matris, tablo, çarpım tablosu) için kullanılır; işlem sayısı genellikle n^2 veya n^3'tür.
- C++ dilinde döngüler `for`, `while`, `do-while` anahtar kelimeleriyle yazılır.
- Bu bölümde 15 farklı sayı problemi C++ diliyle çözülmüştür: toplam, faktöriyel, çift sayılar, ters yazdırma, basamak sayısı, asal sayılar, Fibonacci, EBOB/EKOK, mükemmel sayı, Armstrong sayısı,

tahmin oyunu, çarpım tablosu, elmas çizimi, sıralama ve ortalama.
- Sık yapılan hatalardan kaçınmak için döngü değişkenlerinin güncellenmesi, koşulların doğru yazılması ve iç içe döngülerde farklı değişken isimlerinin kullanılması gerekir.

---

10. DEĞERLENDİRME SORULARI

1. Döngü nedir? Neden kullanılır?
2. `for` ve `while` döngüleri arasındaki farkları maddeler hâlinde yazınız.
3. `do-while` döngüsünün diğerlerinden farkı nedir?
4. `break` ve `continue` komutlarının görevlerini örneklerle açıklayınız.
5. İç içe döngü nedir? Bir örnekle gösteriniz.
6. İç içe döngülerde toplam işlem sayısı nasıl hesaplanır?
7. 1'den 50'ye kadar olan sayıların toplamını bulan C++ programını yazınız.
8. Kullanıcıdan alınan N sayısının faktöriyelini hesaplayan C++ programını yazınız.
9. 1'den 100'e kadar olan tek sayıların toplamını bulan C++ programını yazınız.
10. Kullanıcıdan alınan sayının basamakları toplamını bulan C++ programını yazınız.
11. 1'den 100'e kadar olan asal sayıları listeleyen C++ programını yazınız.
12. İlk 20 Fibonacci sayısını ekrana yazdıran C++ programını yazınız.
13. Kullanıcıdan alınan iki sayının EBOB ve EKOK'unu bulan C++ programını yazınız.
14. Kullanıcıdan alınan sayının mükemmel sayı olup olmadığını kontrol eden C++ programını yazınız.
15. Yıldızlarla aşağıdaki deseni çizen C++ programını yazınız (n = 5 için):

```
*
**
***
****
*****
****
***
**
*
```

16. 1'den 10'a kadar çarpım tablosunu iç içe döngü kullanarak yazdıran C++ programını yazınız.
17. Kullanıcıdan alınan 10 sayıyı diziye atıp büyükten küçüğe sıralayan C++ programını yazınız.
18. Sonsuz döngü nedir? Nasıl önlenir? Bir örnekle açıklayınız.
